# Programmazione 2

a.a. 2023-2024

## lista linkata (o concatenata)

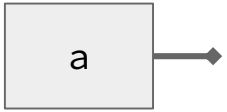
# LISTA LINKATA O CONCATENATA

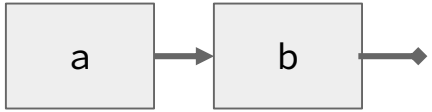
# Lista linkata (concatenata)

In generale **una lista linkata è un elenco**:

★ o è vuota

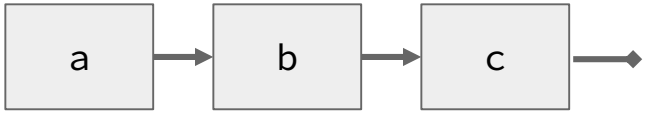
★ o è costituita da un elemento seguito da una lista linkata



# Ruoli degli elementi

A seconda della sua collocazione un elemento ha dei **ruoli**:




**a**:

**primo elemento** della lista

**predecessore** di b

**b**:

**predecessore** di c

**successore** di a

**c**:

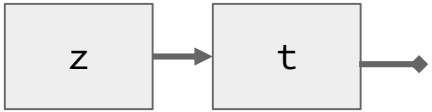
**ultimo elem.** della lista

**successore** di b

# Lista linkata

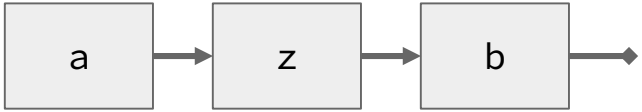
Una **lista linkata:**

★ Può essere ordinata (es. ordine alfabetico inverso)



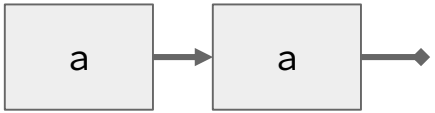

proprietà extra che impatta sulla collocazione degli elementi

★ Può essere non ordinata




nessuna proprietà extra sulla collocazione degli elementi

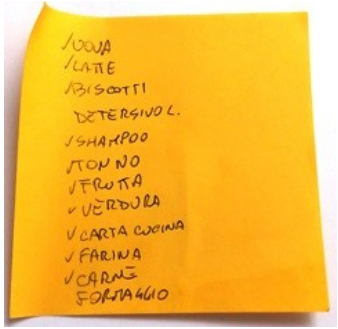
★ Lo stesso dato potrebbe comparire più volte



# Operazioni su liste linkate

- ★ Gli elementi possono essere **inseriti** o **rimossi** all'inizio, alla fine o fra due elementi qualsiasi
- ★ Se la lista è ordinata (presenta una certa proprietà), le operazioni che modificano la lista dovranno **preservare il suo ordinamento** (preservare la proprietà)
- ★ La lista può essere percorsa per **calcolare informazioni** (es. quanti elementi? Somma dei valori contenuti negli elementi in posizione dispari?)

# Lista della spesa




**Esempio**: una lista della spesa è un elenco di prodotti da comperare, **non ordinato** e lo stesso elemento potrebbe **comparire più volte**

Gli elementi possono essere inseriti/rimossi all’inizio, alla fine o anche in mezzo a seconda di come ci sembra meglio o di quando vengono trovati in negozio

# Lista della spesa, aggiunta elementi

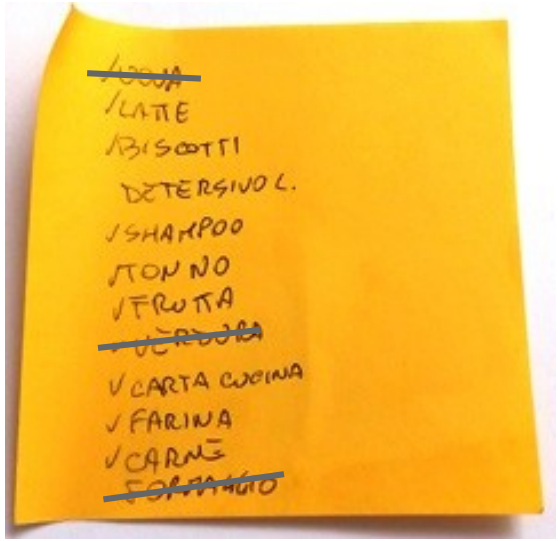
+ TORTA

Mi ricordo che è il compleanno di un’amica

# Lista della spesa, rimozione elementi

Tanti criteri:

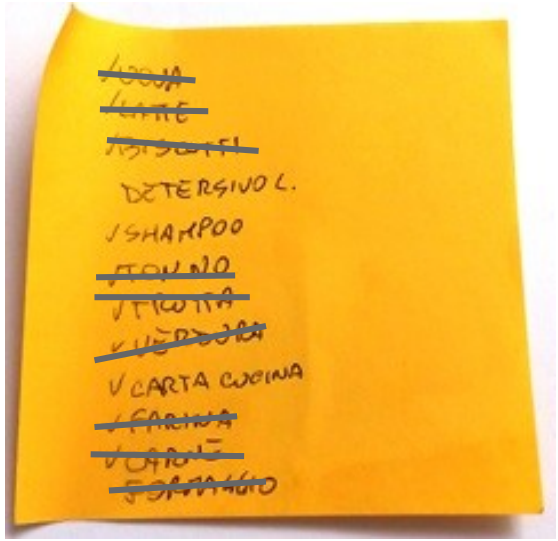
- ★ **Cancello uno o più elementi dati, es. uova, verdura e formaggio**
- ★ Cancello tutti gli elementi che godono di una proprietà, es. tutti gli alimenti
- ★ Cancello l'elemento in una posizione (in testa o in coda)



# Lista della spesa, rimozione elementi

Tanti criteri:

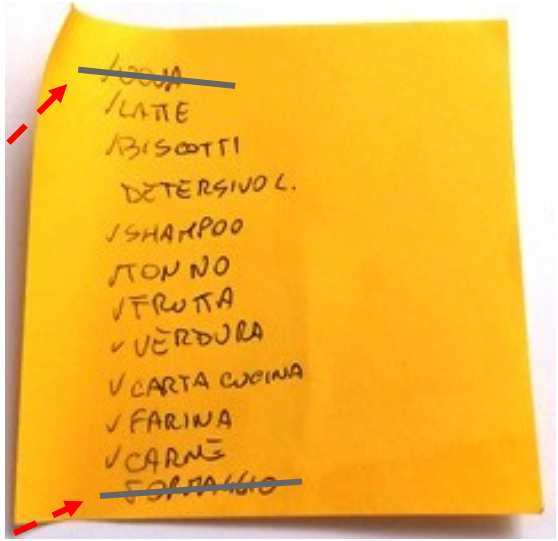
- ★ Cancello uno o più elementi dati, es. uova, verdura e formaggio
- ★ **Cancello tutti gli elementi che godono di una proprietà, es. tutti gli alimenti**
- ★ Cancello l’elemento in una posizione (in testa o in coda)



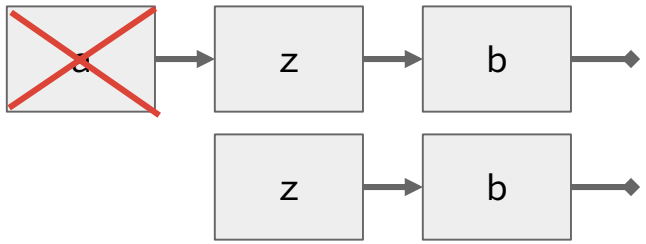
# Lista della spesa, rimozione elementi

Tanti criteri:

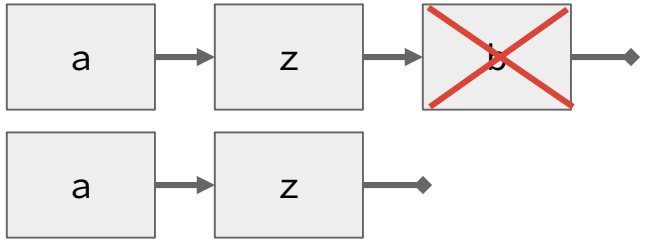
- ★ Cancello uno o più elementi dati, es. uova, verdura e formaggio
- ★ Cancello tutti gli elementi che godono di una proprietà, es. tutti gli alimenti
- ★ **Cancello l'elemento in una posizione (in testa o in coda)**



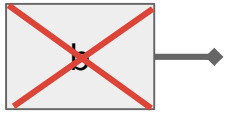
# La cancellazione modifica la lista




z sarà il nuovo primo elemento della lista

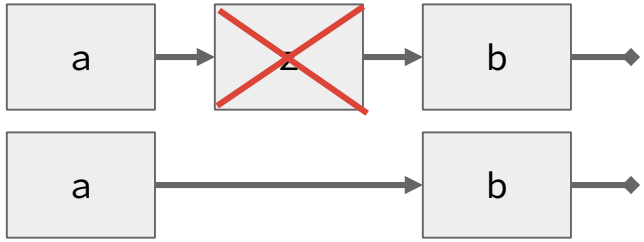



z sarà il nuovo ultimo elemento della lista



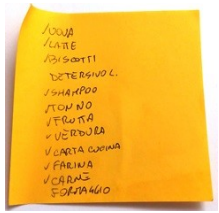

la lista diventa vuota

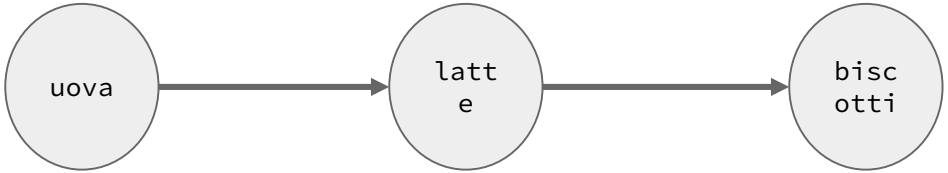
# La cancellazione modifica la lista




occorre collegare i pezzi rimanenti

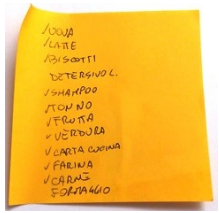
# Lista in un programma

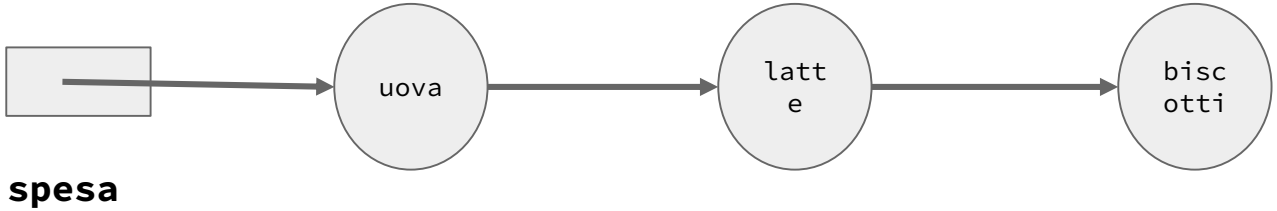



**LISTA LINKATA**: sequenza eventualmente vuota di **elementi**, ogni elemento **ha un riferimento** al successivo

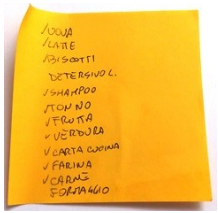
# Lista in un programma




Accesso reso possibile da una variabile

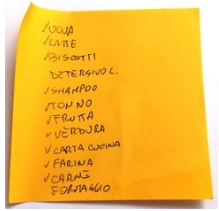
# Lista in un programma



**spesa**

All’inizio la lista è vuota

# Lista in un programma

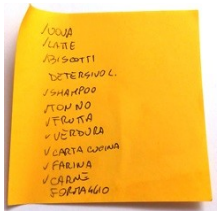


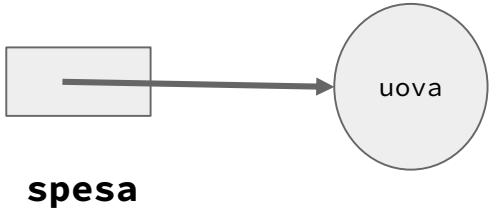
**spesa**

All’inizio la lista è vuota

Ho bisogno di un modo per dire che non ci sono elementi

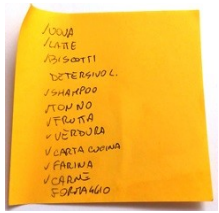
# Lista in un programma

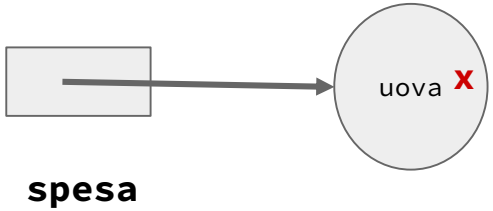



Si inserisce il primo elemento

# Lista in un programma

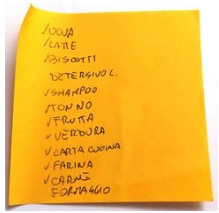


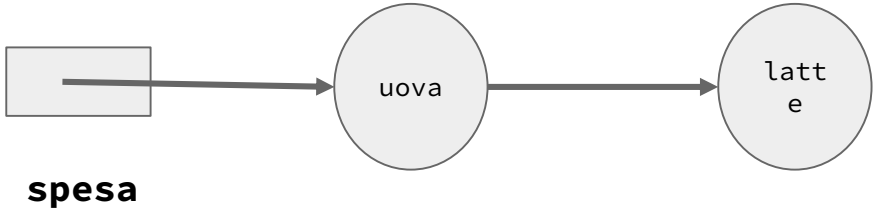




Si inserisce il primo elemento

Devo indicare in qualche modo che questo elemento è anche l’ultimo della lista

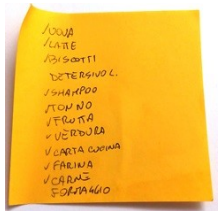
# Lista in un programma

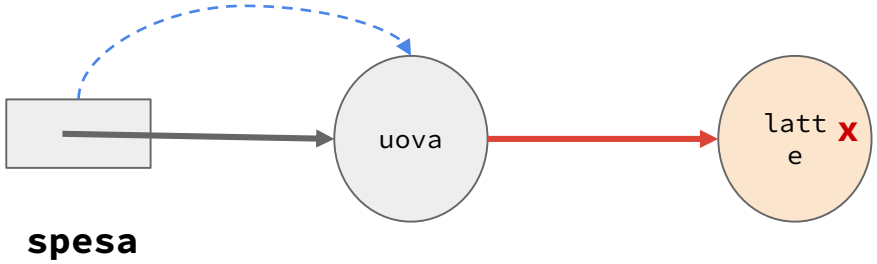



Poi il secondo, per esempio in fondo (in coda)

# Lista in un programma

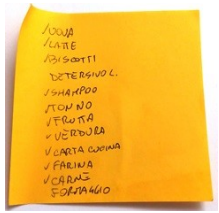


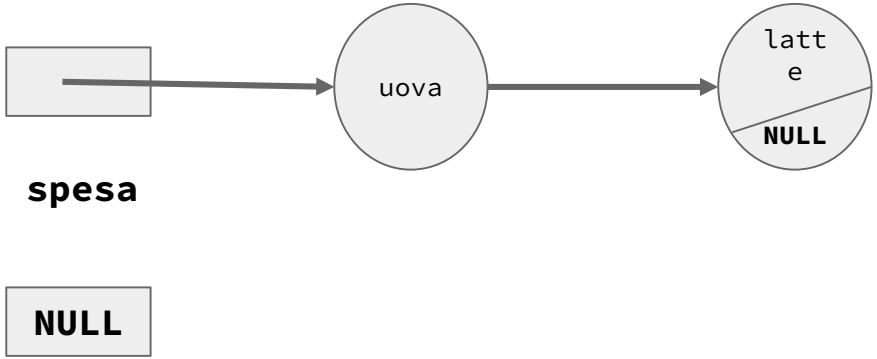




Poi il secondo, per esempio in fondo (in coda):

- ★ Devo accorgermi che la lista NON È VUOTA
- ★ Devo trovare l’ultimo elemento percorrendola
- ★ Devo creare un nuovo elemento e agganciarlo
- ★ Devo indicare che ora questo è l’ultimo

# Lista in un programma

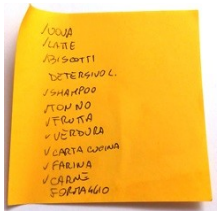


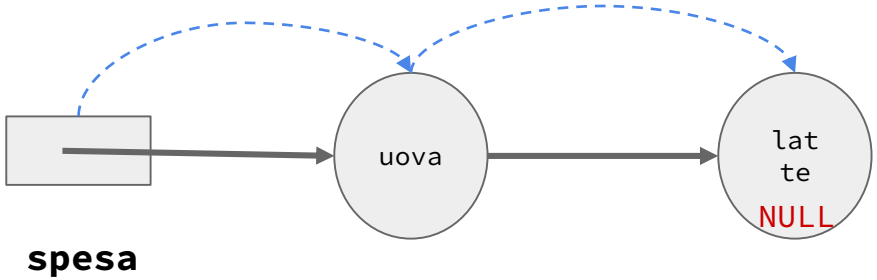




**NULL** indica il termine della lista

È importantissimo!!!

# Lista in un programma

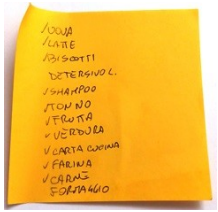


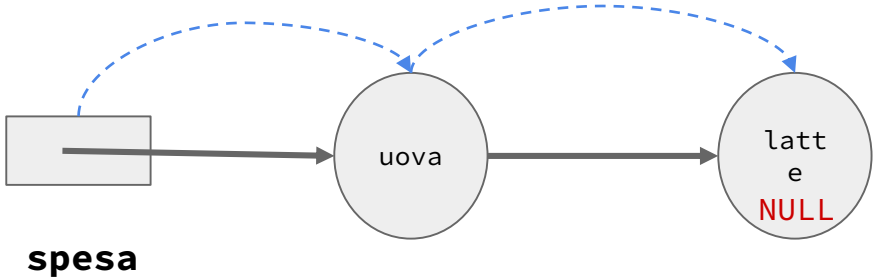



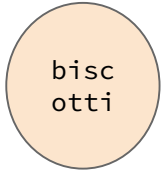

Aggiungiamo anche i biscotti, di nuovo in coda:

- ★ spesa == NULL?
- ★ FALSO, posizionati sul primo elem. della lista (uova)
- ★ Questo elemento è l'ultimo?
- ★ FALSO, spostati sull'elemento successore (latte)
- ★ … continua …

# Lista in un programma



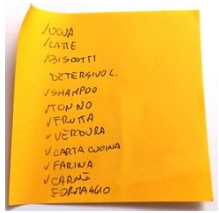



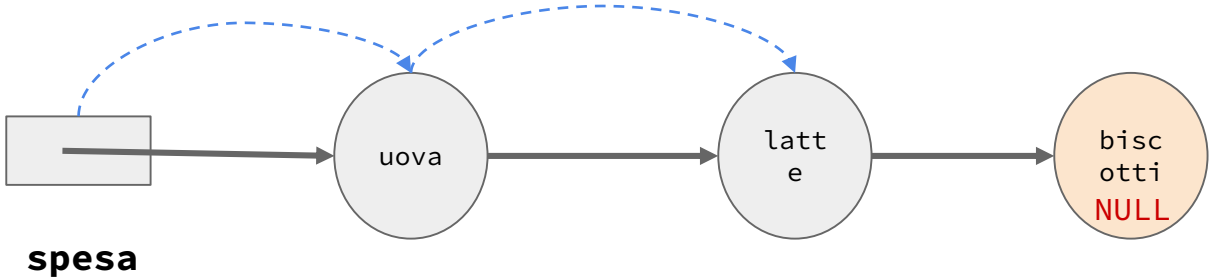




Aggiungiamo i biscotti:

- ★ …
- ★ L’elemento latte è l’ultimo!
- ★ Crea biscotti
- ★ …

# Lista in un programma




Aggiungiamo i biscotti:

- ★ …
- ★ Aggancia biscotti dopo a latte
- ★ Aggiungi che biscotti è ora l’ultimo elemento

# Variabile lista

Una **lista** è una variabile che contiene un riferimento a un nodo oppure NULL

# Nodi di una lista concatenata

Un **nodo** contiene due informazioni eterogenee:

1. un **dato** (es. uova, biscotti)
2. un **riferimento al nodo successore**, eventualmente NULL

Il successore sarà dello stesso tipo del nodo considerato

# Nodi di una lista concatenata

I dati possono essere molti, es:

1. **dato 1**, prodotto da comperare (es. uova)
2. **dato 2**, quantità (es. 12)
3. un **riferimento al nodo successore**, eventualmente NULL

| |
|---|
| uova |
| 12 |
| NULL |

# Nodi di una lista

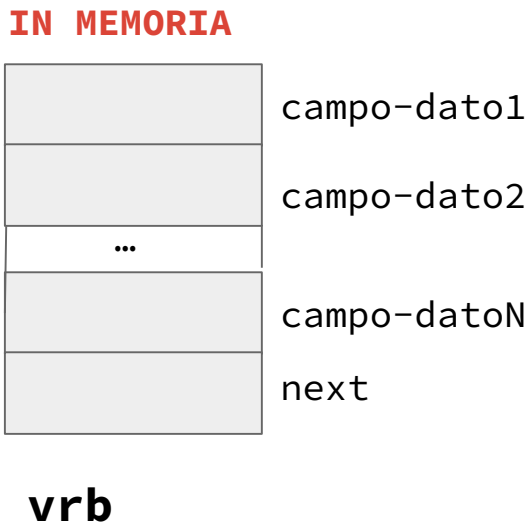
```
struct nodo {
        tipo-dato1   campo-dato1;
        tipo-dato1   campo-dato1;
    …
        tipo-dato1   campo-dato1;
```

**successore? Indirizzo di un nodo che sarà generato in futuro. Posso indicarlo come “struct nodo*”?**

```
}
```

# Nodi di una lista, idea #1

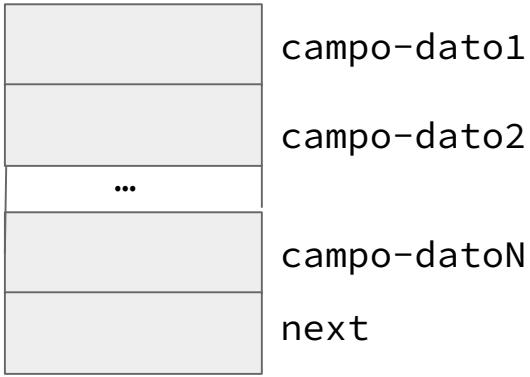
```
struct nodo {
        tipo-dato1   campo-dato1;
        tipo-dato2   campo-dato2;
    …
        tipo-datoN   campo-datoN;
    struct nodo* next;
};
struct nodo vrb;
```



# Nodi di una lista, idea #1

```
struct nodo {
        tipo-dato1   campo-dato1;
        tipo-dato2   campo-dato2;
    …
        tipo-datoN   campo-datoN;
   struct nodo* next;
};
struct nodo vrb;
```

IN MEMORIA




**vrb**

**LISTA VUOTA?**
**SPRECO SPAZIO**

# Nodi di una lista, idea #2

```
struct nodo {
        tipo-dato1  campo-dato1;
        tipo-dato2  campo-dato2;
    …
        tipo-datoN  campo-datoN;
   struct nodo* next;
};
struct nodo *vrb;
```

# Nodi di una lista, idea #2

```
struct nodo {
        tipo-dato1   campo-dato1;
        tipo-dato2   campo-dato2;
    …
        tipo-datoN   campo-datoN;
   struct nodo* next;
};
struct nodo *vrb;
```

**IN MEMORIA**

NULL

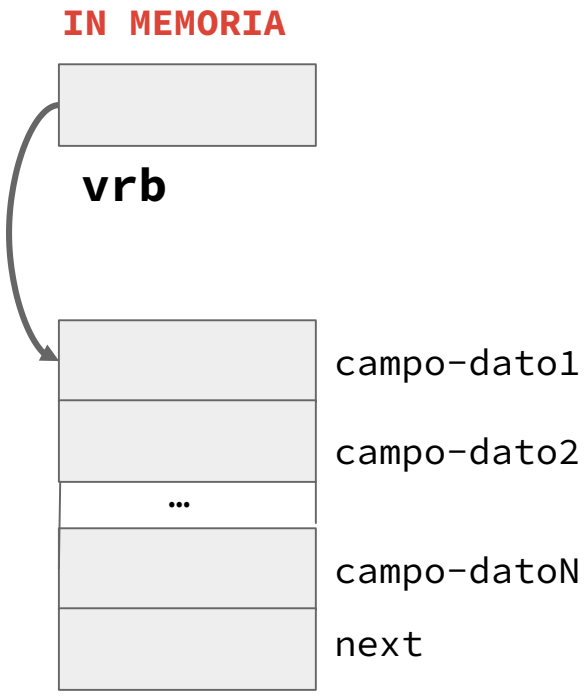
**vrb**

**LISTA VUOTA**

vrb = NULL;

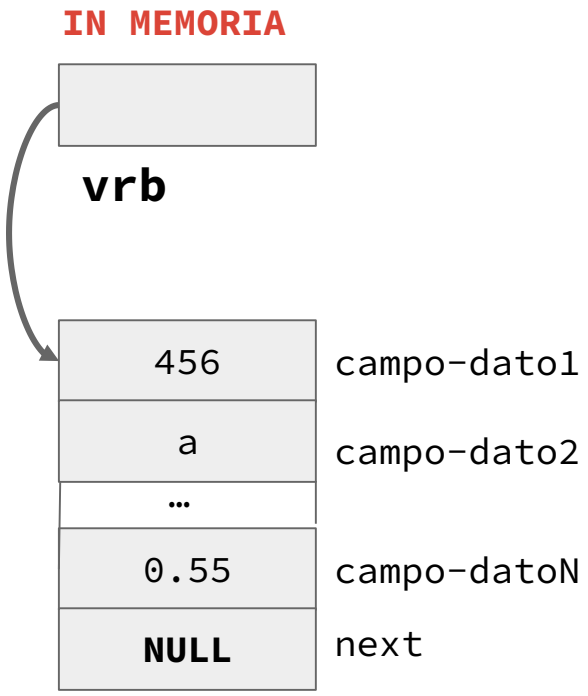
# Nodi di una lista, idea #2

```
struct nodo {
            tipo-dato1   campo-dato1;
            tipo-dato2   campo-dato2;
    …
            tipo-datoN   campo-datoN;
   struct nodo* next;
};
struct nodo *vrb;
vrb = (struct nodo*)
            malloc(sizeof(struct nodo));
```



# Nodi di una lista, idea #2

```
struct nodo {

        …

};

struct nodo *vrb;

vrb = (struct nodo*)
            malloc(...);

vrb->campo-dato1 = expr;

… // inizializza

vrb->next = NULL;
```



# Nodi di una lista, idea #3

```
typedef struct nodo* lista;
struct nodo {
        tipo-dato1  campo-dato1;
        tipo-dato2  campo-dato2;
    …
        tipo-datoN  campo-datoN;
    lista next;
};
lista vrb;
```

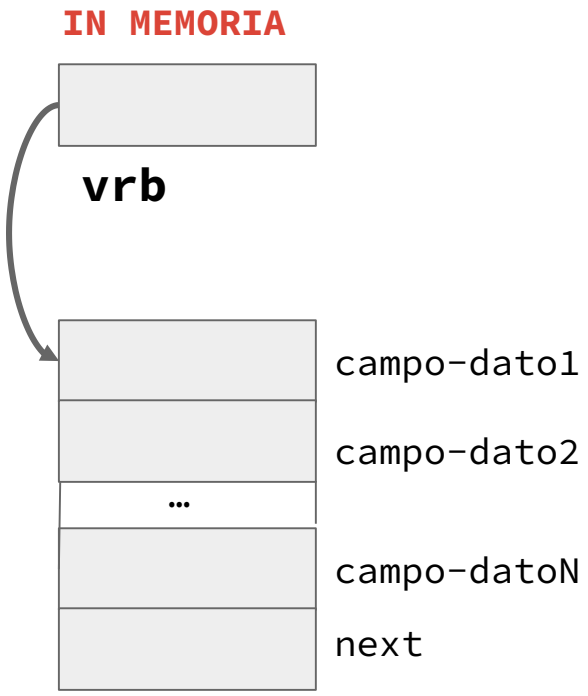
IN MEMORIA

**vrb**

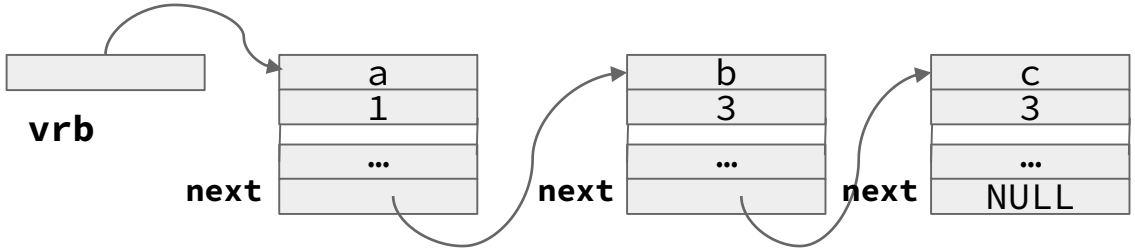
# Nodi di una lista, idea #3

nodo: si inizializza come nel caso precedente

```
typedef struct nodo* lista;
struct nodo {
            tipo-dato1   campo-dato1;
            tipo-dato2   campo-dato2;
      …
            tipo-datoN   campo-datoN;
    lista next;
};
lista vrb;
vrb = (lista)malloc(sizeof(struct nodo));
```



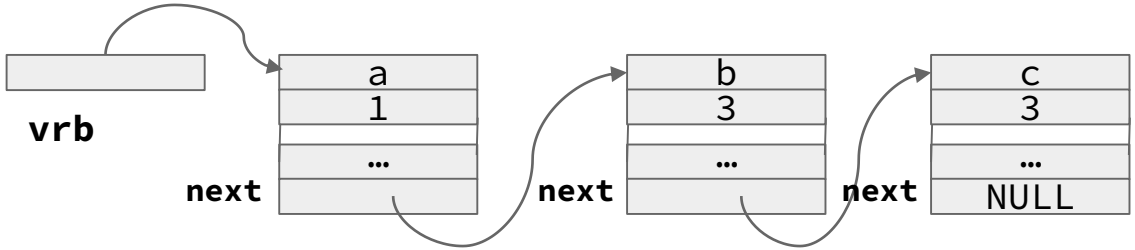
# Accesso agli elementi




```
vrb->campo1

vrb->next->campo1

vrb->next->next->campo2

vrb->next->next->next

vrb->next->next->next->campo1
```

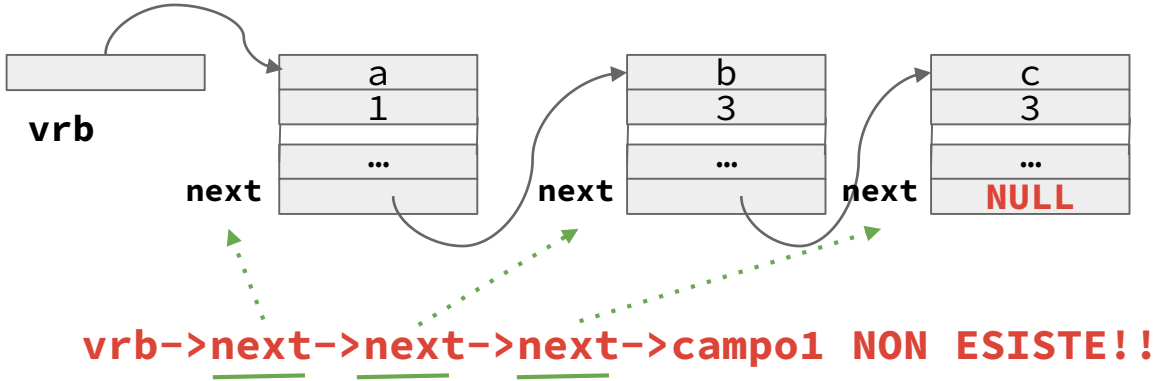
QUANTO VALGONO?

# Accesso agli elementi



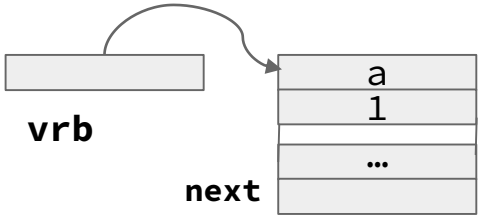

```
vrb->campo1 == a

vrb->next->campo1 == b

vrb->next->next->campo2 == 3

vrb->next->next->next == NULL
```

# ATTENZIONE




Cerco di accedere
a un elemento
che non esiste

# ATTENZIONE 2

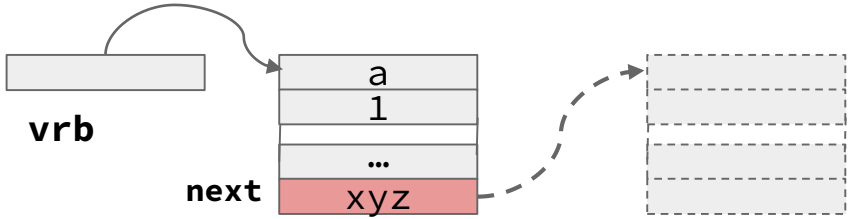



cosa succede quando mi dimentico di impostare **next** a **NULL**?

**vrb->next->campo1**

CHE VALORE HA?

# ATTENZIONE 2




tentativo di accesso a un nodo fantasma

next corrisponde a una locazione di memoria contenente **bit casuali** che vengono **interpretati come un indirizzo**

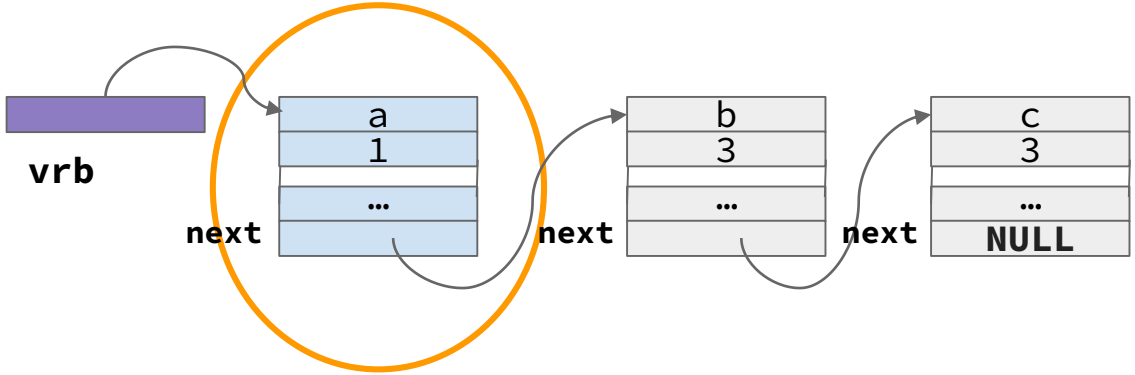
**vrb->next->campo1**

vuole accedere a un campo di un **nodo non allocato**

tipicamente il processo viene terminato (segmentation fault)

# Operazioni di base su una lista linkata

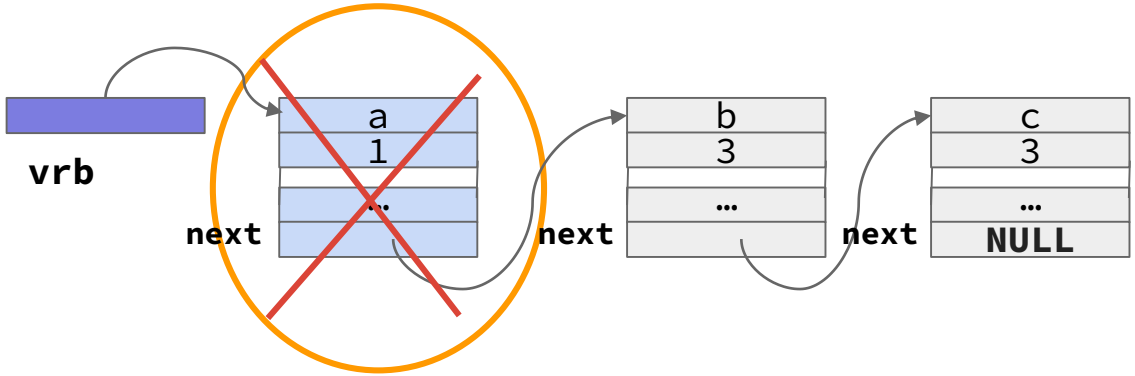
# TESTA DI UNA LISTA




**Testa: primo elemento accessibile**

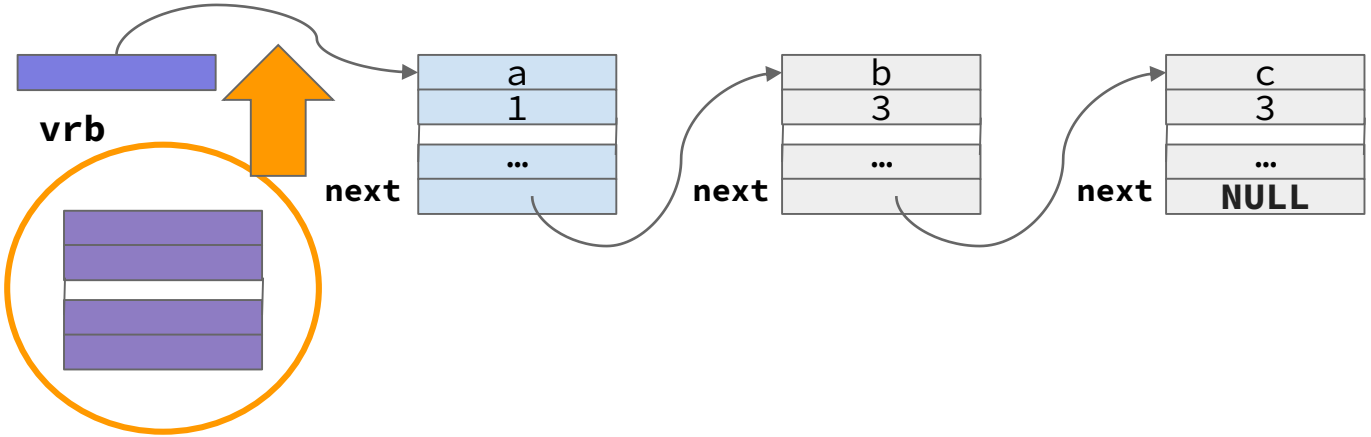
- ★ Rimozione dell’elemento in testa
- ★ Aggiunta di un elemento in testa

# Modificare la testa di una lista




- ★ Rimozione dell’elemento in testa:
  - il successore diventa il primo elemento
  - se il successore è NULL la lista diventa vuota

# Modificare la testa di una lista

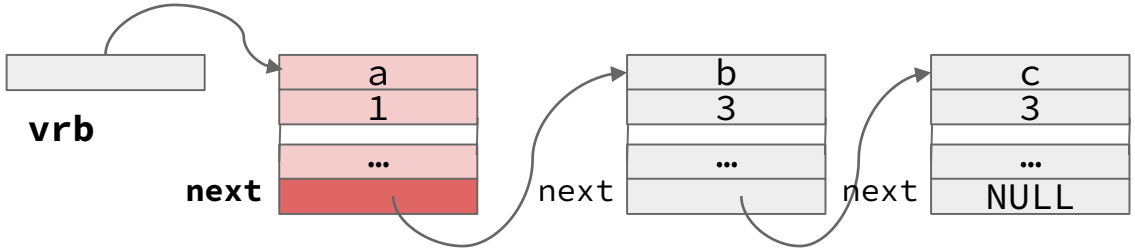



★ Aggiunta di un elemento in testa:

- il precedente primo elemento diventa il successore di quello nuovo

# Cancellazione dalla testa

# Cancellazione dalla testa



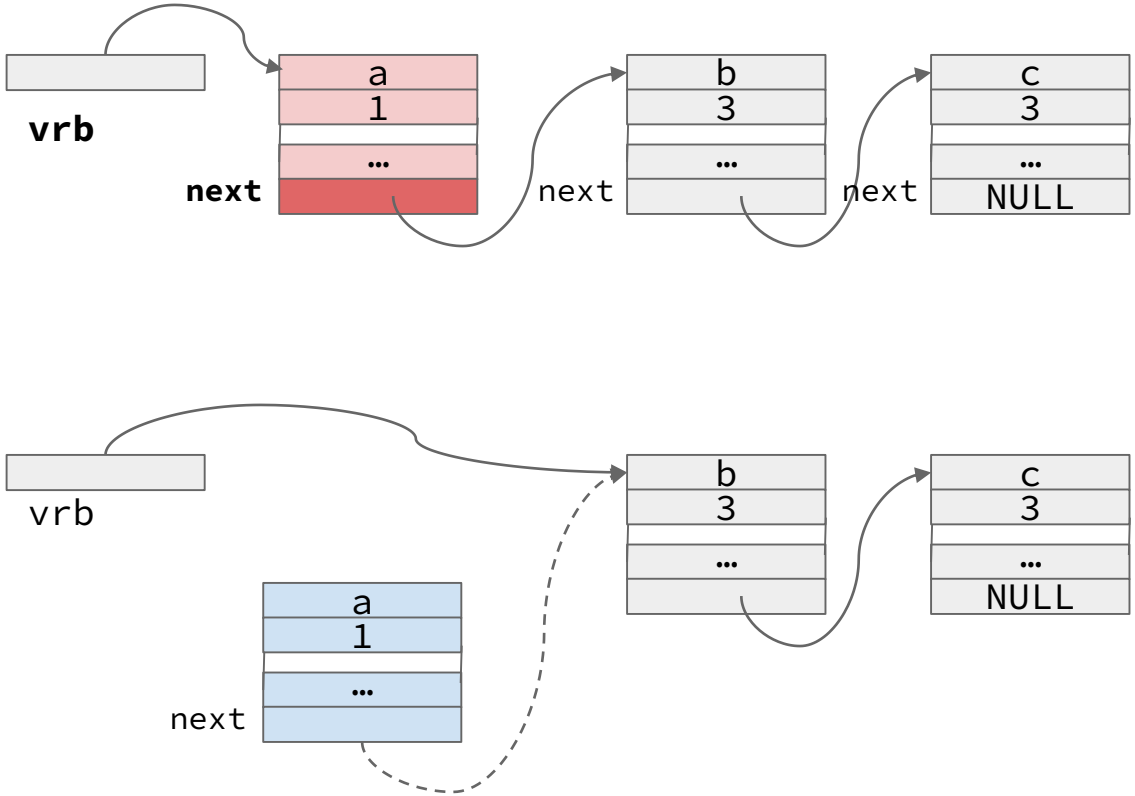

**EFFETTO:** vrb deve puntare all'attuale secondo elemento

Il secondo elemento è ricordato nel campo **NEXT** del primo a cui si accede tramite vrb

IDEA! Basta fare:

```
vrb = vrb->next;
```

# Cancellazione dalla testa




**EFFETTO:** vrb deve puntare all'attuale secondo elemento

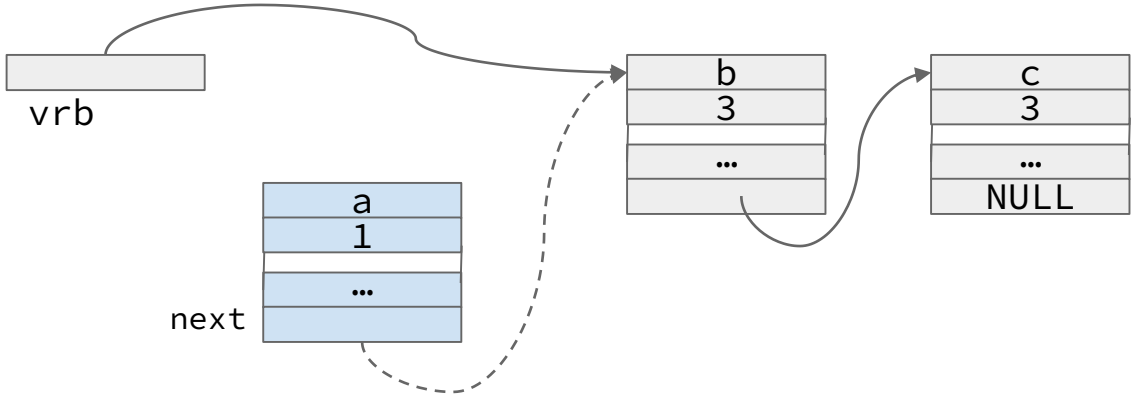
Il secondo elemento è ricordato nel campo **NEXT** del primo a cui si accede tramite vrb

IDEA! Basta fare:

```
vrb = vrb->next;
```

**NO**

# Cancellazione dalla testa




- non più raggiungibile
- non più utilizzabile

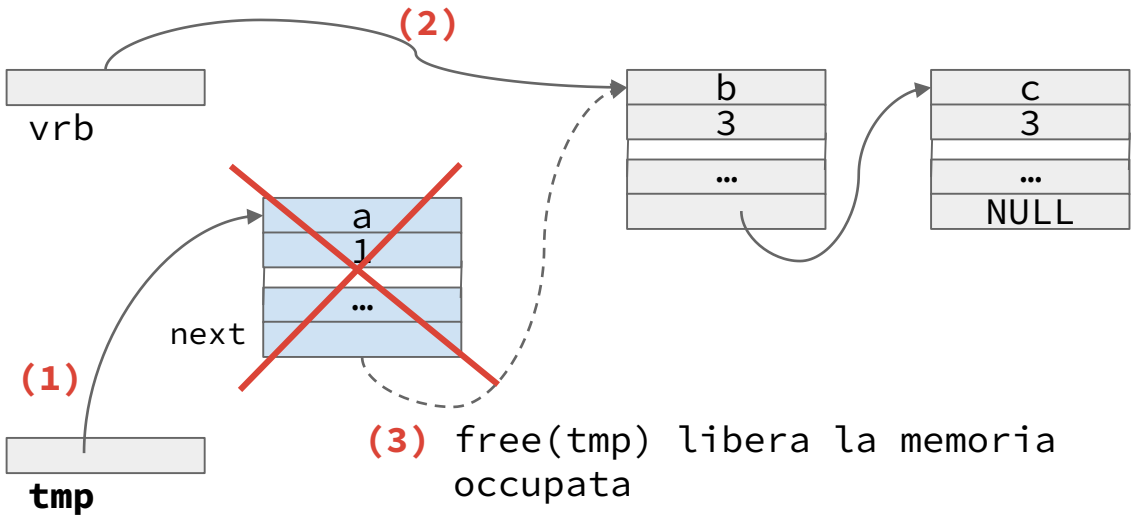
MA

- memoria risulta occupata
- non assegnabile per contenere altro

**ATTENZIONE:**
l'elemento è ancora presente nello heap (in memoria) ma non è più raggiungibile

Il programma spreca memoria. **Questo è un errore grave**

# Cancellazione dalla testa




**tmp**: variabile in più, di comodo mantiene vivo l'accesso alla struttura

**ATTENZIONE**:
l'elemento è ancora presente nello heap (in memoria) ma non è più raggiungibile

**Codice corretto:**

```
lista tmp;

 1) tmp = vrb;
 2) vrb = vrb->next;
 3) free(tmp);
```

# Funzione per cancellare dalla testa, V1

```
lista ML;

… // crea i nodi

ML = cancella-testa(ML);
```

**cancella-testa**: funzione che, data una lista, restituisce la stessa lista privata del primo elemento; la funzione fa anche la free di tale elemento

**NB**: questa soluzione richiede la collaborazione dell'utente; ML è modificata dall'assegnamento, non da cancella-testa

# Funzione per cancellare dalla testa, V1

```
lista ML;
… // crea i nodi
ML = cancella-testa(ML);
```

NB: questa soluzione richiede la collaborazione dell'utente (l'**assegnamento** modifica ML)

---

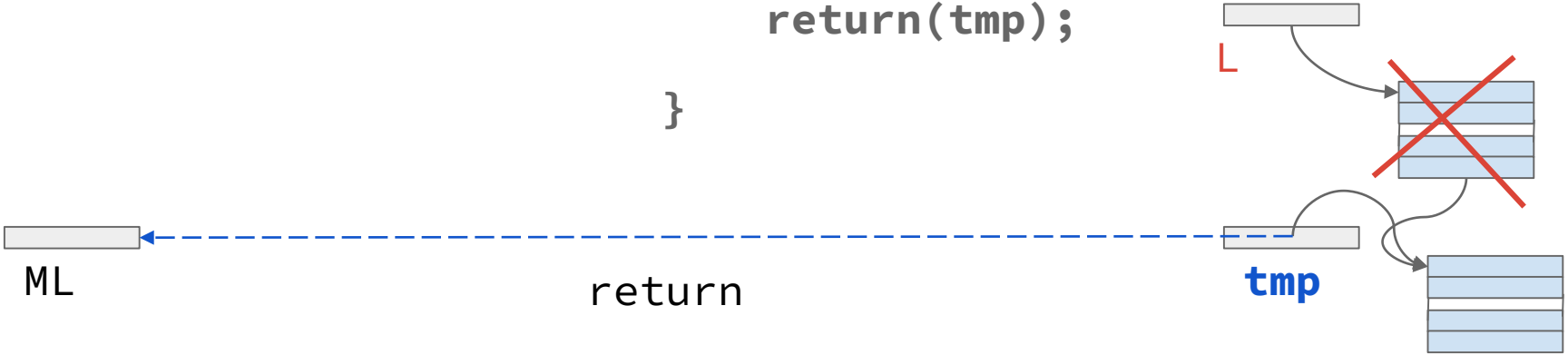
```
int x;
… // dai un valore a x
x = sottrai-uno(x);
```

È affine, strutturalmente, al codice che vedete per modificare un intero (la funzione restituisce il valore di x-1)

# Funzione per cancellare dalla testa, V1

```
lista ML;
… // crea i nodi
ML = cancella-testa(ML);
```

```
lista cancella-testa(lista L) {
    lista tmp = L->next;
    free(L);
    return(tmp);
}
```



# Funzione per cancellare dalla testa, v2

```
lista ML;

… // crea i nodi

cancella-testa2(&ML);
```

Non voglio richiedere all’utente un assegnamento

Voglio una funzione in grado di modificare essa stessa una variabile che appartiene all’ambiente chiamante

Devo passare la lista per riferimento

# Funzione per cancellare dalla testa, v2

```
lista ML;
… // crea i nodi
cancella-testa2(&ML);
```

Non voglio richiedere all'utente un assegnamento

---

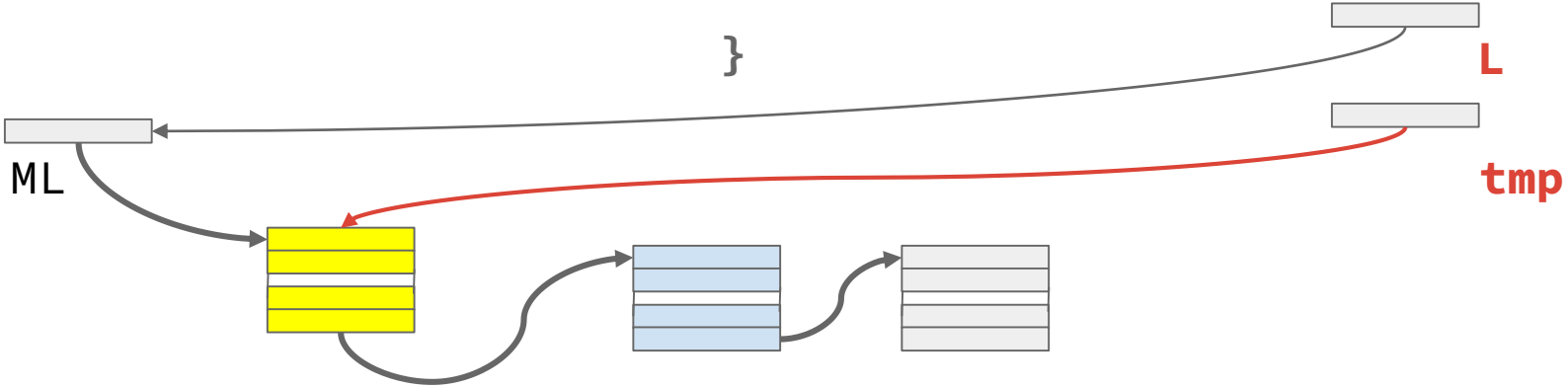
```
int x;
… // dai un valore a x
sottrai-uno(&x);
```

è affine a questo codice in cui sottrai-uno stessa assegna un valore alla variabile dell'ambiente chiamante

# Funzione per cancellare dalla testa, v2

```
lista ML;
… // crea i nodi
cancella-testa2(&ML);
```

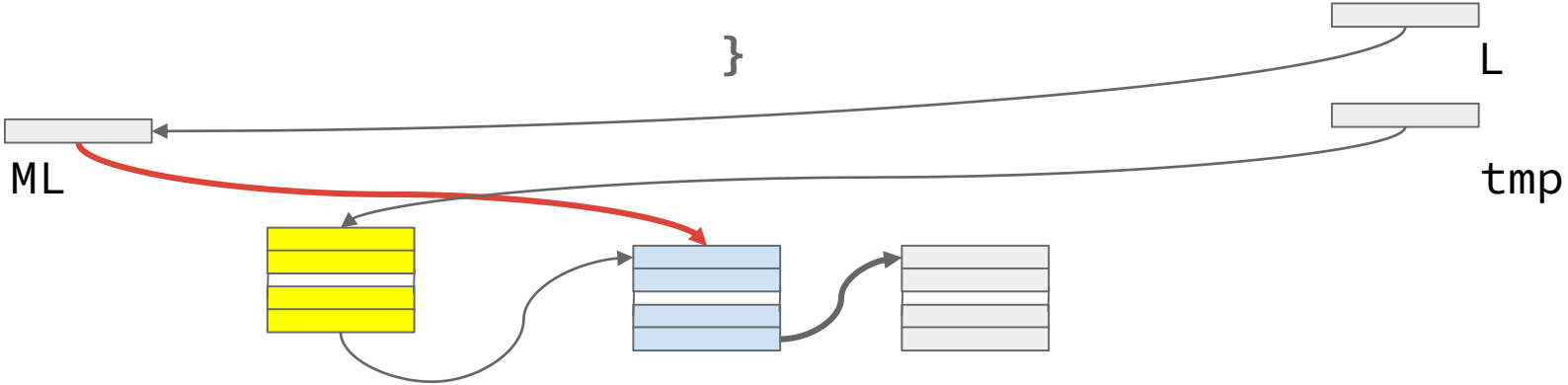
```
void cancella-testa2(lista *L) {
    lista tmp = (*L);
    *L = (*L)->next;
    free(tmp);
}
```



# Funzione per cancellare dalla testa, v2

```
lista ML;
… // crea i nodi
cancella-testa2(&ML);
```

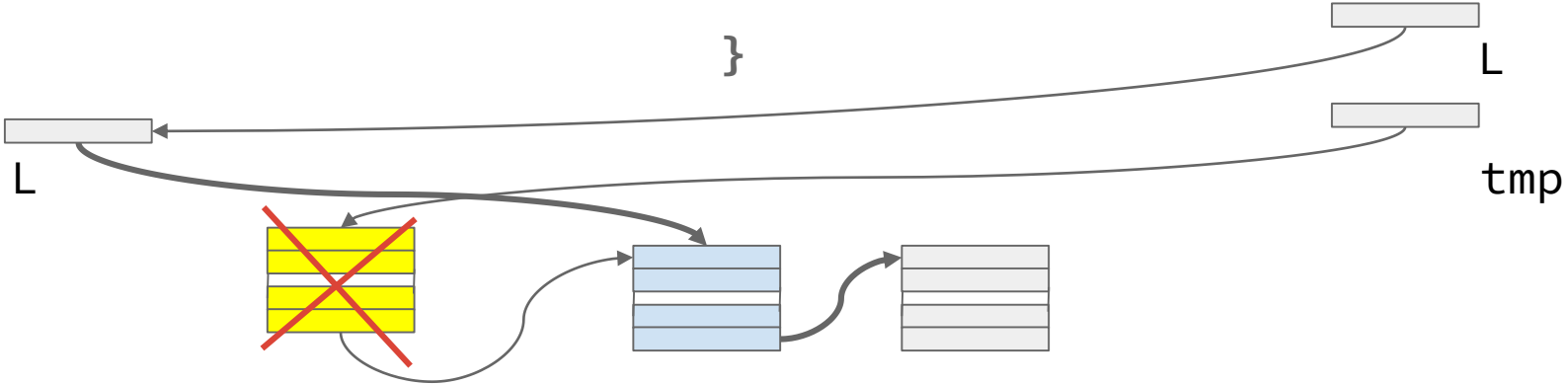
```
void cancella-testa2(lista *L) {
    lista tmp = (*L);
    *L = (*L)->next;
    free(tmp);
}
```



# Funzione per cancellare dalla testa, v2

```
lista ML;
… // crea i nodi
cancella-testa2(&ML);
```

```
void cancella-testa2(lista *L) {
    lista tmp = (*L);
    *L = (*L)->next;
    free(tmp);
}
```



# DETTAGLIO: PERCHÉ MODIFICO *L E NON L?

```
*L = (*L)->next;
```

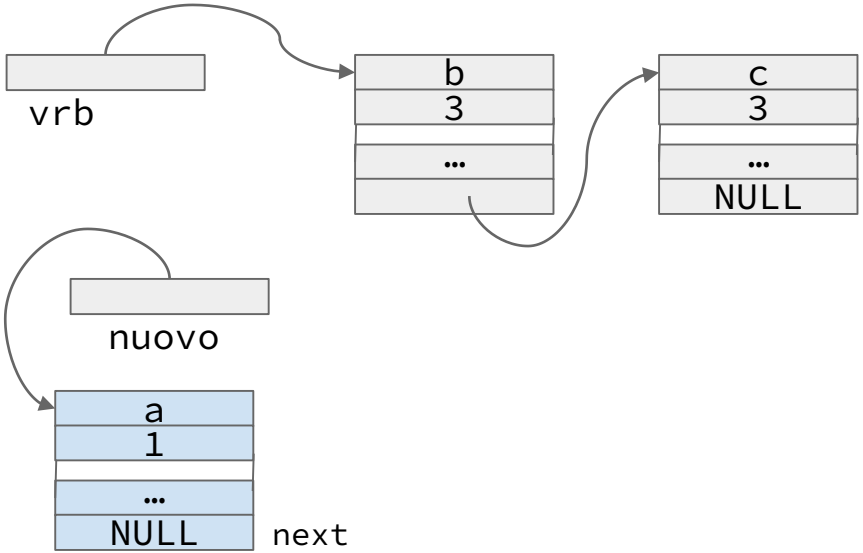
oppure

```
L = L->next;
```

?

# Inserzione in testa

# Inserzione in testa




elemento da inserire
come primo

nuovo: variabile di comodo

vrb è una lista linkata

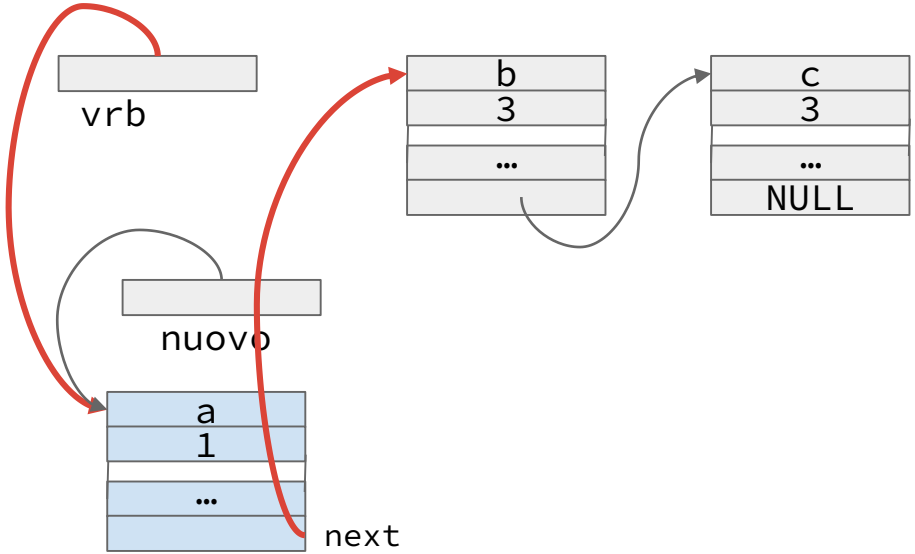
**nuovo è il riferimento a un nuovo elemento, creato e inizializzato:**

```
nuovo = (lista)malloc(
                sizeof(struct nodo));

nuovo->… = … // assegna valori

nuovo->next = NULL;
```

# Dettaglio: crea nodo

```
lista crea-nodo( … dati …) {
    lista nuovo = (lista)malloc(sizeof(struct nodo));
    nuovo->… = … // assegna i valori ai campi dati
    nuovo->next = NULL;
    return nuovo;
}
```

# Inserzione in testa




nuovo elemento da inserire come primo

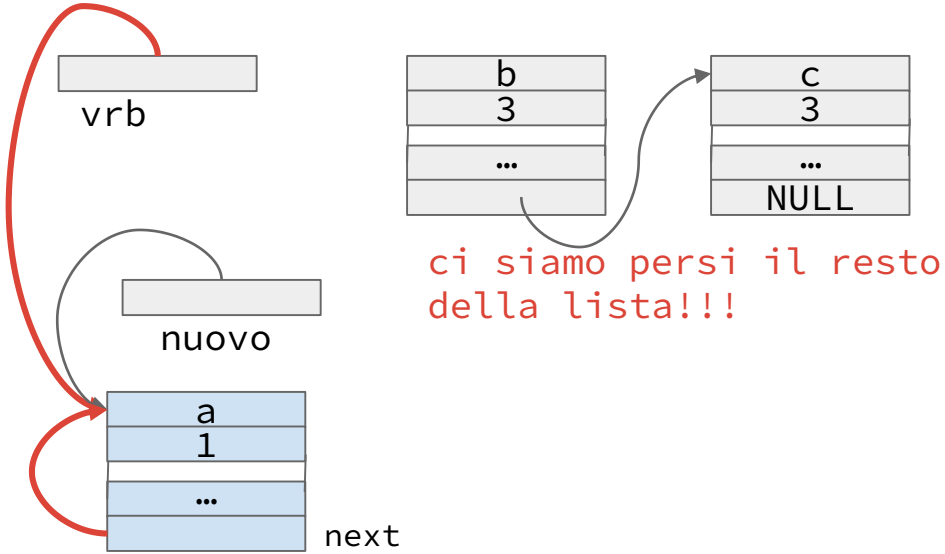
**nuovo** dovrà puntare all'elemento attualmente puntato da **vrb**, poi **vrb** punterà a **nuovo**.

Due assegnazioni:

**(1) nuovo->next = vrb;**

**(2) vrb = nuovo;**

# L'ordine è importante!!



**Sbagliare l'ordine è un errore gravissimo:**

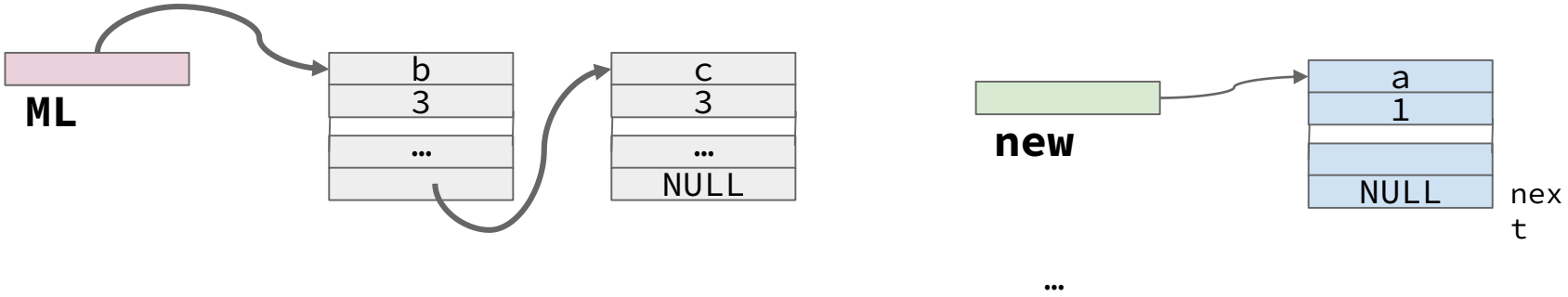
**(1) vrb = nuovo;**

**(2) nuovo->next = vrb;**

il successore del nuovo elemento sarà quello che ha per indirizzo il contenuto di vrb, cioè l'elemento stesso

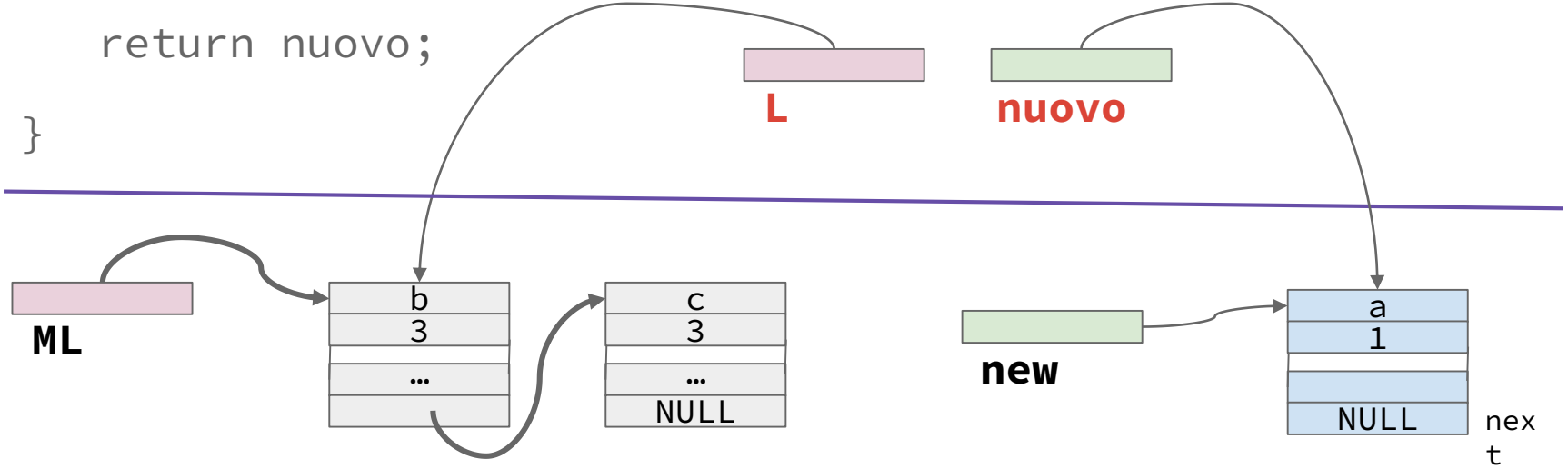
# Funzione per inserzione in testa, V1

```
lista ML, new;
… // dopo un po'
new = (lista) malloc(sizeof(struct nodo));
ML = aggiungi_in_testa(ML, new);
```



# Funzione per inserzione in testa, V1

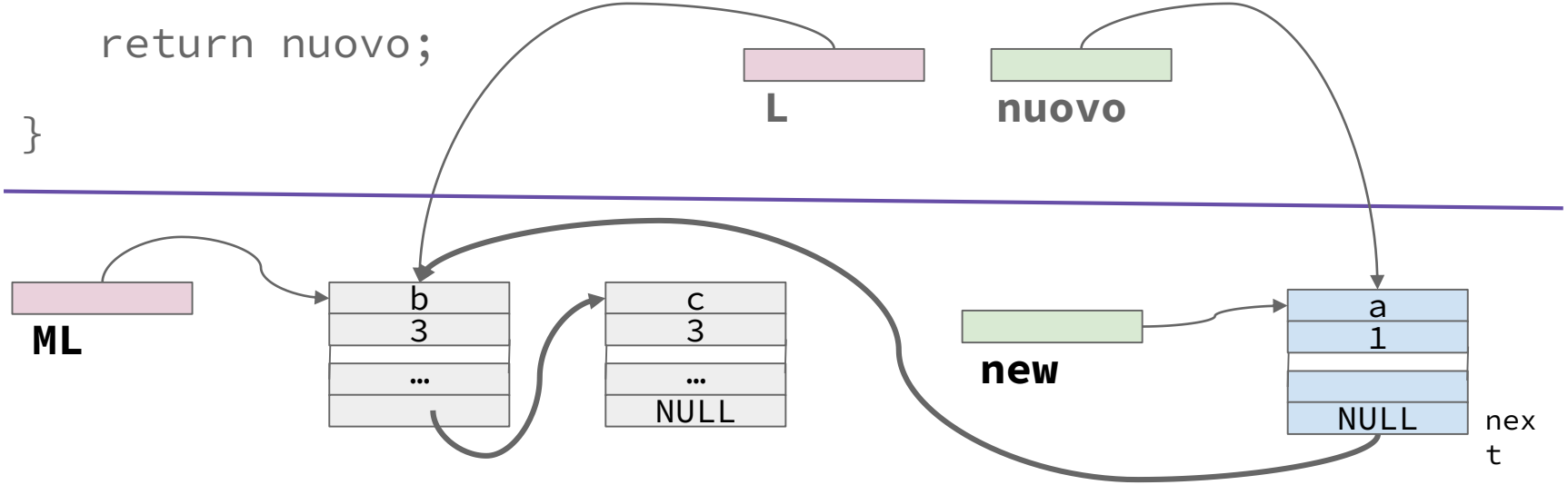
```
lista aggiungi_in_testa(lista L, lista nuovo) {
    nuovo->next = L;
    return nuovo;
}
```




AMBIENTE CHIAMANTE

# Funzione per inserzione in testa, V1

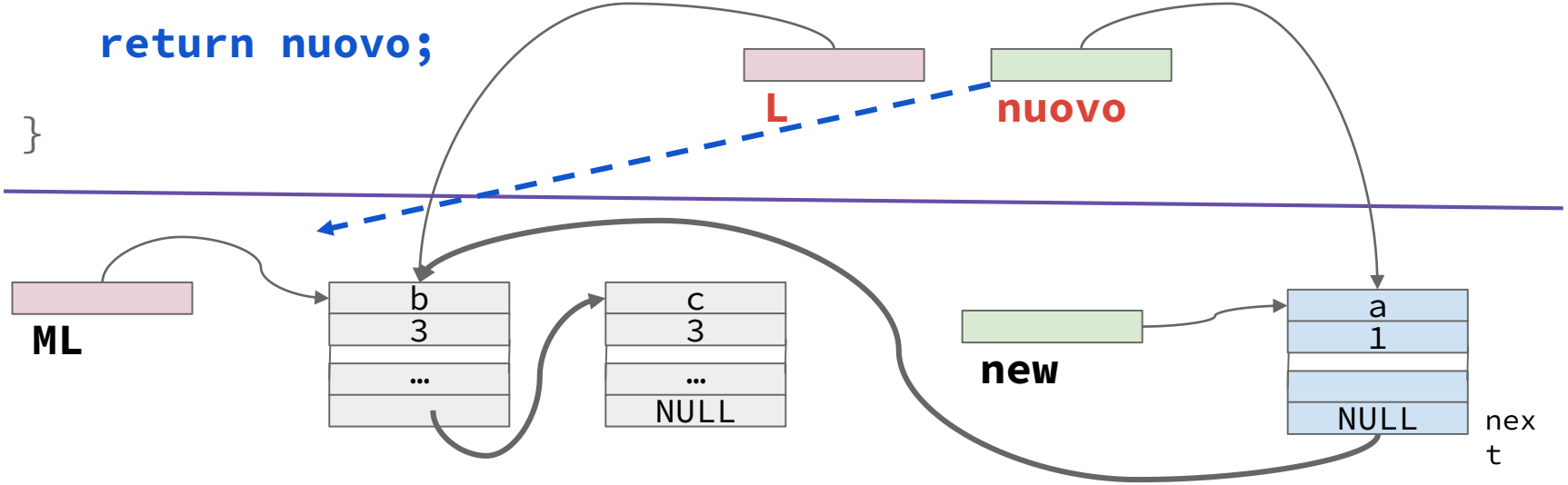
```
lista aggiungi_in_testa(lista L, lista nuovo) {
    nuovo->next = L;
    return nuovo;
}
```




AMBIENTE CHIAMANTE

# Funzione per inserzione in testa, V1

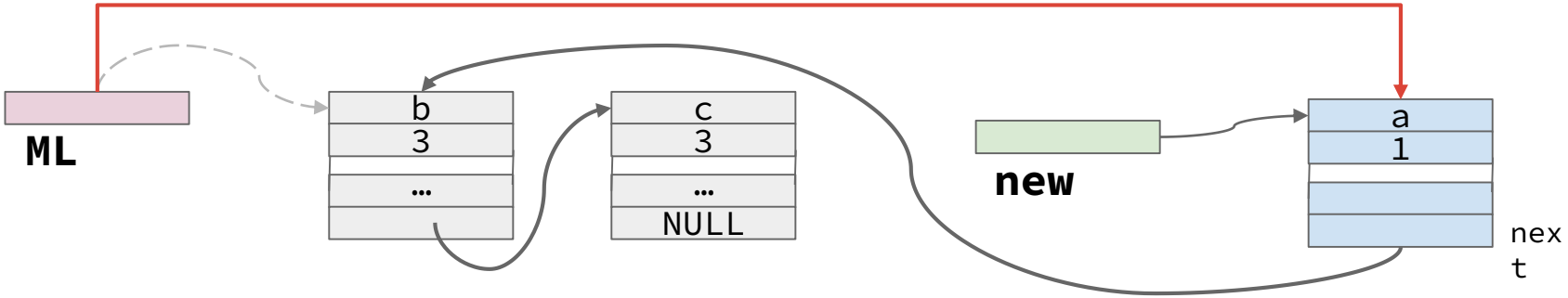
```
lista aggiungi_in_testa(lista L, lista nuovo) {
    nuovo->next = L;
    return nuovo;
}
```




AMBIENTE CHIAMANTE

# Funzione per inserzione in testa, V1

```
lista ML, new;
… // dopo un po'
new = (lista) malloc(sizeof(struct nodo));
ML = aggiungi_in_testa(ML, new);
```



# Funzione per inserzione in testa, v2

```
lista ML, new;
… // dopo un po’
new = (lista) malloc(sizeof(struct nodo));
aggiungi_in_testa2(&ML, new);
```

# Funzione per inserzione in testa, v2

```
aggiungi_in_testa2(lista *L, lista nuovo) {
    nuovo->next = *L;
    *L = nuovo;
}
```

```
main() {
    lista ML;
    …
    aggiungi_in_testa2(&ML, new);
    …
}
```

# Lista linkata: altre funzioni

- ★ inserzione in coda (ultima posizione)
- ★ cancellazione dalla coda (ultima posizione)
- ★ inserire in posizione generica
- ★ cancellare da posizione generica
- ★ percorrimento: scorro la lista per raccogliere informazioni (es. quanti elementi soddisfano una certa proprietà …) o per svolgere un'operazione su ogni elemento (es. stampa)
- ★ accesso misto (es. percorrimento e cancellazione di tutti gli elementi in posizione pari)

**NB:** DA SAPER IMPLEMENTARE NELLE DUE VERSIONI (V1,V2)

# Lista linkata: altre funzioni

Noi vedremo insieme solo alcuni casi, fare da soli gli altri

- ★ inserzione in coda (ultima posizione)
- ★ cancellazione dalla coda (ultima posizione)
- ★ inserire in posizione generica
- ★ cancellare da posizione generica
- ★ percorrimento: scorro la lista per raccogliere informazioni (es. quanti elementi soddisfano una certa proprietà …) o per svolgere un'operazione su ogni elemento (es. stampa)
- ★ accesso misto (es. percorrimento e cancellazione di tutti gli elementi in posizione pari)

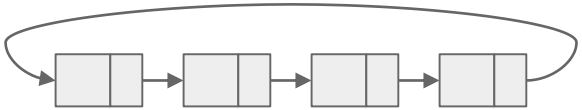
# Tipi di liste

La lista linkata vista è una base per realizzare liste di vari tipi:

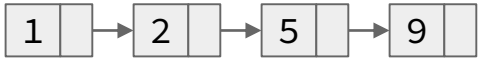
- ★ lista circolare
- ★ lista ordinata (più in generale: a priorità)
- ★ lista double ended
- ★ lista doppiamente linkata

# Tipi di liste

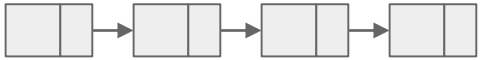
★ lista circolare:



★ lista ordinata:




★ lista double ended:



si può inserire/estrarre ad ambo le estremità

★ lista doppiamente linkata: